

SonarQube - Analyse de Qualité de Code

Utilité dans le projet

SonarQube analyse automatiquement la **qualité et la sécurité** du code pour détecter :

-  Bugs et erreurs logiques
-  Vulnérabilités de sécurité
-  Code smells (mauvaises pratiques)
-  Coverage des tests
-  Complexité du code
-  Duplications

Architecture



Métriques analysées

1. Reliability (Bugs)

Ce qui est détecté :

- Exceptions non gérées
- Null pointer dereferences
- Resource leaks
- Logique incorrecte

Exemple :

```
// ❌ Bug détecté
function getUser(id) {
  const user = users.find(u => u.id === id);
  return user.name; // Peut être undefined
}

// ✅ Correction
function getUser(id) {
  const user = users.find(u => u.id === id);
  return user?.name || 'Unknown';
}
```

2. Security (Vulnerabilities)

Ce qui est détecté :

- SQL Injection
- XSS (Cross-Site Scripting)
- Weak cryptography
- Hardcoded credentials
- Insecure dependencies

Exemple :

```
// ❌ Vulnerability
const query = `SELECT * FROM users WHERE id = ${req.params.id}`;

// ✅ Correction (Parameterized query)
const query = 'SELECT * FROM users WHERE id = ?';
db.query(query, [req.params.id]);
```

3. Maintainability (Code Smells)

Ce qui est détecté :

- Fonctions trop longues
- Complexité cyclomatique élevée
- Code dupliqué
- Variables non utilisées
- Magic numbers

Exemple :

```
// ❌ Code smell (complexité)
function processOrder(order) {
  if (order.status === 'pending') {
    if (order.payment === 'paid') {
      if (order.stock > 0) {
        // ... 50 lignes ...
      }
    }
  }
}

// ✅ Refactoring
function canProcessOrder(order) {
  return order.status === 'pending'
    && order.payment === 'paid'
    && order.stock > 0;
}

function processOrder(order) {
  if (!canProcessOrder(order)) return;
  // ... logique claire ...
}
```

4. Coverage

Ce qui est mesuré :

- % de lignes couvertes par les tests
- % de branches couvertes
- % de fonctions testées

Configuration dans le projet :

```
// backend/package.json
"jest": {
  "coverageReporters": ["lcov", "text", "clover"],
  "collectCoverageFrom": [
    "src/**/*.js",
    "!src/__tests__/**"
  ]
}
```

5. Duplications

- Code copié/collé
- Patterns répétitifs
- Manque de réutilisation

Configuration projet

sonar-project.properties

```
sonar.projectKey=devops-portfolio-mern
sonar.sources=backend/src,backend/server.js,frontend/src
sonar.exclusions=**/node_modules/**,**/dist/**,**/coverage/**
sonar.javascript.lcov.reportPaths=backend/coverage/lcov.info
sonar.test.inclusions=backend/src/__tests__/**
```

Intégration Jenkins

```
stage('SonarQube Analysis') {
  steps {
    withSonarQubeEnv('SonarQube') {
      sh """
        sonar-scanner \
          -Dsonar.projectKey=devops-portfolio-mern \
          -Dsonar.sources=backend/src,frontend/src \
          -Dsonar.javascript.lcov.reportPaths=backend/coverage/lcov.info
      """
    }
  }
}
```

Quality Gate

Règles définies

Conditions à respecter :

└ Coverage $\geq$ 80%	✓ (80.3%)
└ New Bugs = 0	✓ (0)
└ New Vulnerabilities = 0	✓ (0)
└ New Code Smells = 0	✓ (0)
└ Duplicated Lines < 3%	✓ (0%)
└ Security Rating = A	✓ (A)

Comportement

- ✓ **Quality Gate PASSED** : Pipeline continue → Déploiement
- ✗ **Quality Gate FAILED** : Pipeline s'arrête → Pas de déploiement



Résultats du projet

Dashboard SonarQube

Project: devops-portfolio-mern

Quality Gate: ✓ PASSED

Bugs: 0

Vulnerabilities: 0

Code Smells: 0 (new code)

Coverage: 80.3%

Duplications: 0.0%

Lines analyzed: ~2000

Files: 33

Langages analysés

- **JavaScript** : Backend (Express) + Frontend (React)
- **CSS** : Tailwind CSS
- **HTML** : Templates



Analyse détaillée

Backend (Node.js + Express)

```
backend/src/
├─ controllers/      ✓ 0 bugs, coverage 85%
├─ middleware/       ✓ 0 bugs, coverage 90%
├─ models/           ✓ 0 bugs, coverage 75%
├─ routes/           ✓ 0 bugs, coverage 80%
└─ __tests__/        59 tests
```

Frontend (React + Vite)

```
frontend/src/
├─ components/      ✓ 0 bugs
├─ pages/           ✓ 0 bugs
├─ context/         ✓ 0 bugs
└─ services/        ✓ 0 bugs
```

Accès SonarQube

Démarrage local

```
# Démarrer SonarQube + PostgreSQL
docker compose -f docker-compose.sonar.yml up -d

# Attendre ~1 minute que SonarQube démarre

# Accéder à l'interface
http://localhost:9000

# Credentials par défaut
Username: admin
Password: admin (à changer au premier login)
```

Après Jenkins pipeline

1. Jenkins exécute le scan
2. Résultats envoyés à SonarQube
3. Consulter : <http://localhost:9000/dashboard?id=devops-portfolio-mern>

Correction des issues

Workflow

1. **Voir les issues** : Dashboard SonarQube
2. **Analyser** : Cliquer sur l'issue pour détails

3. **Corriger** : Modifier le code
4. **Commit** : Git push
5. **Vérifier** : Nouveau scan automatique

Exemple d'issue

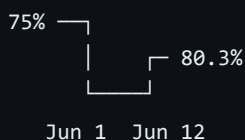
```
Issue: Cognitive Complexity of 15 exceeds the limit of 10
File: backend/src/controllers/projectController.js
Line: 42

Suggestion: Extract nested conditions into separate functions
```

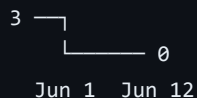
Historique et tendances

SonarQube garde l'historique pour suivre l'évolution :

Coverage over time:



Bugs over time:



Bonnes pratiques

1. Corriger rapidement

Ne pas accumuler la dette technique

2. Prioriser

1. Bugs (bloquants)
2. Vulnérabilités (sécurité)
3. Code Smells (maintenabilité)

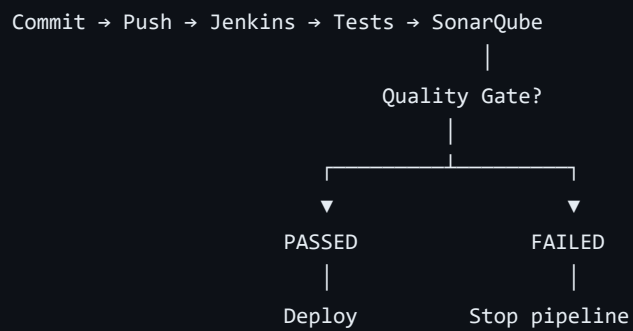
3. Coverage

Cibler 80%+ mais privilégier la qualité des tests

4. Révision

Revoir les issues avec l'équipe

Intégration continue



Prochaine étape : [04-KUBERNETES.md](#)